

Modelling the Efficiency of a Sheep Pasture in Minecraft

Evan Freeland

Introduction

Minecraft is an open-world sandbox video game*. The game takes place in a procedurally generated world made of cubic voxels, or *blocks*. Among other activities, players can engage in agriculture, including the raising of sheep. Sheep can be sheared to gather wool, and must eat grass to regrow their wool after shearing. Grass blocks are converted to dirt after being eaten, and may be converted back to grass by other adjacent grass blocks. If a pasture is packed too densely with sheep, they will deplete the grass, and wool production will be low. However, if the pasture is not densely packed, wool production will still be low because there are not many sheep. Between these two extremes, there is an optimal sheep population that will maximize wool output.

Deterministic model

Minecraft operates largely on chance. Both the growth of grass and the appetite of sheep is determined by random number generation. Every time the game's physics engine performs an update, grass has a chance to spread and sheep have a chance to eat. The behavior of the pasture is therefore governed by stochastic processes. Modelling stochastic processes is no simple task, so we will make a first attempt at modelling by assuming the pasture evolves according to deterministic, continuous-time processes.

Grass mechanics The *Minecraft* physics engine operates in discrete increments of time, called *ticks*. Every tick, a sheep has a chance to attempt to eat the block it's standing on. If it's standing on grass, it will successfully eat the grass and convert it into dirt. If the sheep and grass in the pasture are uniformly distributed, the expected number of sheep standing on grass is the product of the number of sheep and the fraction of grass blocks in the pasture. The rate at which grass is eaten is therefore given by:

$$\mu = \frac{NRN_g}{A} \quad (1)$$

Where μ is the rate at which grass is eaten, A is the size of the pasture, N_g is the number of grass blocks in the pasture, N is the number of sheep, and R is the frequency with which an individual sheep attempts to eat.

The rate at which grass grows not as simple. If N_g is small, then grass growth will be limited by the lack of places the grass can spread *from*. If N_g is large, grass growth will be limited by a lack of places for the grass to spread *to*. To capture this behavior, we will employ a logistic model, where population follows the differential equation:

$$\dot{y} = ky \left(1 - \frac{y}{p} \right)$$

*This investigation was conducted in Minecraft version 1.19.4.

Where $y(t)$ is the population at time t , k is a parameter representing the reproduction rate of the population and p is the carrying capacity, the maximum population that the local environment can sustain.

The logistic model is one of the oldest models for population growth. It was born from the older exponential (or *Malthusian*) model, but modified to include a maximum population size. It is useful as a tool of instruction, but has since been succeeded by more sophisticated and comprehensive models. However, we are not dealing with real-world ecology, but a video game which has simple population mechanics. Therefore the model is appropriate for our purposes. Let's define λ as the grass growth rate. Our population of interest is the number of grass blocks in the pasture N_g , and the carrying capacity is A :

$$\lambda = kN_g \left(1 - \frac{N_g}{A}\right) \quad (2)$$

Where k is a constant as described above.

The rate of population change is the difference of λ and μ :

$$\dot{N}_g = \lambda - \mu = kN_g \left(1 - \frac{N_g}{A}\right) - \frac{NRN_g}{A} \quad (3)$$

At steady state, $\dot{N}_g = 0$. Solving (3) for N_g when $\dot{N}_g = 0$ gives the steady state grass population:

$$N_g = A - \frac{NR}{k} \quad (4)$$

The fast farmer For simplicity, let's imagine the farmer has superhuman speed, and is able to shear each sheep the instant it grows its wool. In this case, the rate at which wool is gathered is equal to μ . Denoting the rate of wool gathering with F , and substituting (4) into (1):

$$F_{fast} = \mu = NR - \frac{N^2 R^2}{Ak} \quad (5)$$

We can find the optimal sheep population by differentiating (5) w.r.t. N , setting the derivative to 0 and solving for N :

$$N = \frac{Ak}{2R} \quad (6)$$

This is the sheep population which will produce the maximum wool output, if the farmer is significantly fast enough. At the optimal value of N , the rate of wool production is:

$$F_{fast} = \frac{Ak}{4}$$

The realistically slow farmer *Minecraft* players typically don't want to put all of their time into shearing their sheep as quickly as they can. For a more realistic model, we'll assume the farmer shears all sheep at once, intermittently and at regular intervals.

The farmer is off doing other things, and every T seconds, they come back to shear the sheep. Upon shearing the sheep, all the sheep are wool-less. By the time they come back for the next shearing, the sheep have regrown some amount of wool. Let's say the number of sheep that have their wool is N_w . So when the farmer shears the sheep, they get N_w wool. Then the average rate at which the farmer gathers wool is

$$F_{slow} = \frac{N_w}{T}$$

Now we need to consider the value of N_w . A sheep will only regrow wool upon eating if it doesn't already have any, so we should expect that the rate at which sheep are regrowing wool is the rate at which they are eating, μ , times the fraction of sheep that are wool-less:

$$\dot{N}_w = \frac{N - N_w}{N} \mu = (N - N_w) \frac{RN_g}{A}$$

At time $t = 0$, the farmer shears all the sheep, leaving them with $N_w = 0$, and then at time $t = T$, the farmer comes back and shears them again, gathering N_w blocks of wool. How many sheep grew their wool back in time T ? We can find out by solving the above equation for N_w . We'll separate it into N_w terms and non N_w terms:

$$\frac{1}{N - N_w} dN_w = \frac{RN_g}{A} dt$$

And integrate from initial state ($t = 0, N_w = 0$) to the final state ($t = T, N_w = N_{w,T}$). We're assuming the pasture has reached steady state, so N_g is independent of t and can be treated as a constant.

$$\int_0^{N_{w,T}} \frac{1}{N - N_w} dw = \int_0^T \frac{RN_g}{A} dt$$

$$-\ln\left(\frac{N - N_{w,T}}{N}\right) = \frac{TRN_g}{A}$$

And solve for $N_{w,T}$:

$$N_{w,T} = N - Ne^{-\frac{TRN_g}{A}}$$

So when the farmer comes back to shear the sheep at $t = T$, they will get $N - N \exp\left(-\frac{TRN_g}{A}\right)$ blocks of wool. If the farmer keeps coming back every T seconds, they will be gathering wool at a rate

$$F_{slow} = \frac{N - Ne^{-\frac{TRN_g}{A}}}{T}$$

Finally, we can plug (4) for N_g :

$$F_{slow} = \frac{N - Ne^{-TR + \frac{TR^2 N}{Ak}}}{T} \quad (7)$$

This is the expression for the rate of wool gathering by a slow farmer. To find the optimal sheep population, we can take the same approach as we did with the fast farmer case. Differentiating (7) w.r.t. N and setting $F_{slow} = 0$:

$$\frac{dF_{slow}}{dN} = \frac{1 - \left(\frac{TR^2 N}{Ak} + 1\right) e^{-TR + \frac{TR^2 N}{Ak}}}{T} = 0$$

And solving for N :

$$N = \frac{Ak}{TR^2} (W(e^{TR+1}) - 1) \quad (8)$$

Where W is the Lambert W function, which satisfies $W(xe^x) = x$. Plugging this back into (7):

$$F_{slow} = \frac{Ak(W(e^{TR+1}) - 1)(1 - e^{-TR + W(e^{TR+1}) - 1})}{R^2 T^2}$$

Experimental determination of rate constants Before proceeding further, it's worth determining the actual values of k and R . This will be done via in-game experiments, which will be described briefly here. More detail can be found on the author's github[†].

Ten sheep were locked in cells with a grass block immediately underneath them. The cells included circuitry to count when each sheep ate the grass and automatically replenish it. In one hour, the sheep consumed 320 blocks of grass. This gives an R value of 0.00888 s^{-1} .

A 30x30 patch of dirt was isolated from its surroundings and uniformly interspersed with 100 grass blocks. As the grass grew, the grass population was recorded every 5 seconds. The data was fit to the exact solution of the logistic equation:

$$y = \frac{p}{1 + \left(\frac{p-y_0}{y_0}\right) e^{-kt}}$$

The best fit gave $k = 0.00773 \text{ s}^{-1}$. The data and fit are compared in Figure 1. The close fit gives us also confidence that the logistic model is appropriate.

Discussion Figure 2 shows F_{fast} and N_g as a function of N , normalized for pasture area. These are simple curves, a parabola and straight line respectively, but there are a few important features to glean. First, the equilibrium N_g and F_{fast} are both zero at $\frac{N}{A} = \frac{k}{R} = 0.869$. Deterministically, a pasture with this density will lose all of its grass.

Second, the maximum F_{fast} occurs at $N_g = 0.5A$. This is a feature of the logistic model; population growth is greatest when the population is half of the carrying capacity.

[†] github.com/ELFREELAND/minecraft_sheep

Figure 1 Logistic model fit to grass growth data.

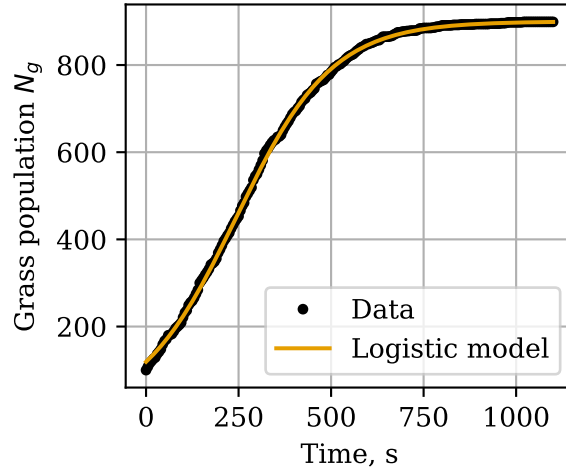


Figure 2 Plot of $\frac{F_{fast}}{A}$ and $\frac{N_g}{A}$ as a function of N/A .

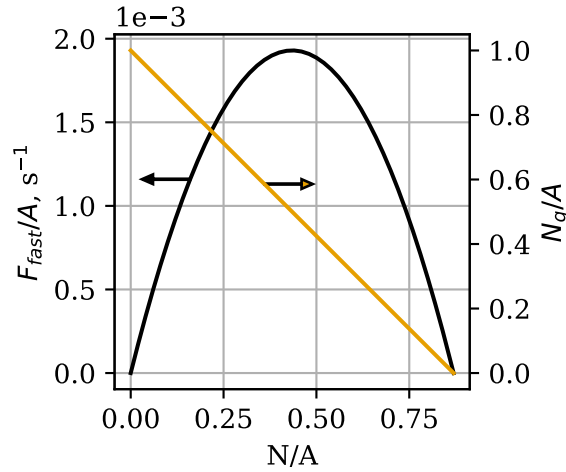


Figure 3 shows F_{slow} as a function of N and T . For the case of $T = 1$, F_{slow} follows the same parabolic pattern with respect to N as F_{fast} . We can see this algebraically by employing the following approximation, for small x :

$$e^x - 1 \approx x$$

In equation (7), the exponent is small for small values of T , therefore:

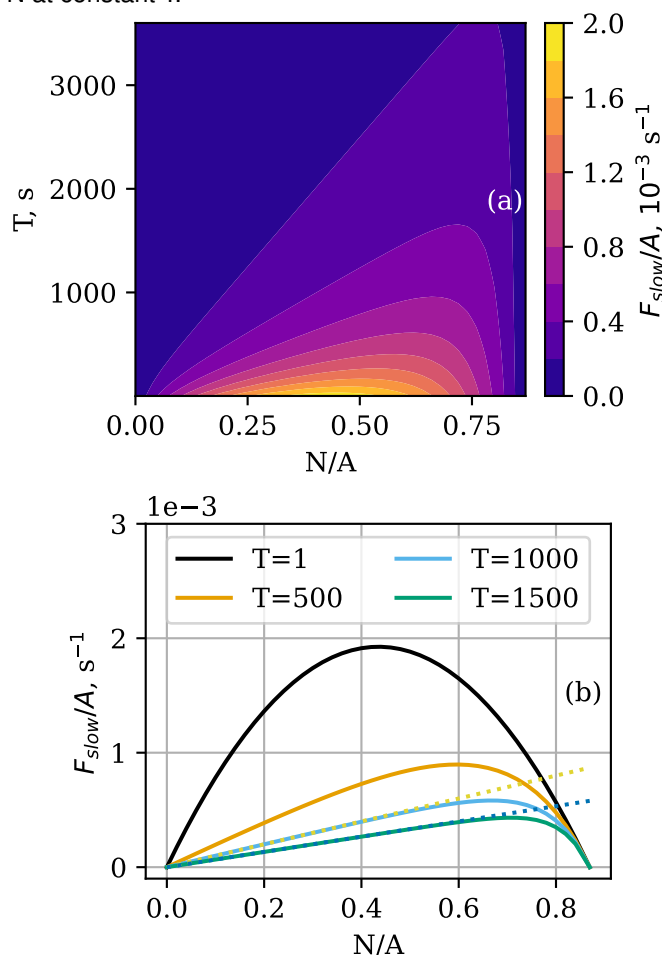
$$F_{slow} \approx NR - \frac{N^2 R^2}{Ak} = F_{fast}$$

For large values of T and modest values of N , F_{slow} is approximately linear w.r.t. N . Algebraically, the term $e^{-TR(1-\frac{RN}{Ak})} \approx 0$ in (7) for large negative values of the exponent. Under these conditions,

$$F_{slow} \approx \frac{N(1-0)}{T} = \frac{N}{T}$$

Physically, this represents conditions for which all of the sheep have regrown their wool by the time the farmer comes back to harvest, so the farmer is simply gathering N blocks of wool with every harvest. Lines of $\frac{N}{T}$ are plotted over top of the curves in figure 3(a).

Figure 3 (a) Heatmap plot of F_{slow} as a function of T and N . (b) Contours of F_{slow} as a function of N at constant T .



Stochastic model

The deterministic model gives valuable insight into the expected behavior of the pasture, but a more complete picture can be obtained by developing a stochastic model which captures the randomness inherent in the physics engine of *Minecraft*. We will develop a stochastic model for the grass population in the pasture using Kolmogorov equations. This approach for population modelling is described in more detail in [1], but the key elements will be outlined here.

Kolmogorov equations In the deterministic model, we considered the grass population as a single number, N_g , which evolved with time. In the stochastic approach, we will consider the evolution of the *probability distribution* of N_g with time.

We'll introduce the following notation:

$$p_x(t) = \Pr(N_g(t) = x)$$

In words, $p_x(t)$ is the probability that the pasture has $N_g = x$ at time t .

We'll also need to slightly reconsider the meanings of μ and λ . In the deterministic model, these are the rate of grass growth or consumption. For a small time interval Δt , the number of grass blocks that are eaten is $\mu\Delta t$, and likewise for λ . If Δt is sufficiently short so that the probability of two independent changes in grass population is negligible, $\mu\Delta t$ can be reinterpreted as the probability of one grass block being consumed in time Δt , and likewise for λ . μ and λ therefore represent the probability of the pasture transitioning to a state with one more or one less grass block than before.

Using these new interpretations of μ and λ , we can write a system of differential equations describing the evolution of the probability distribution of N_g with time:

$$\dot{p}_x(t) = -p_x(t)(\lambda(x) + \mu(x)) + p_{x-1}(t)\lambda(x-1) + p_{x+1}(t)\mu(x+1)$$

Note that $\mu(x)$ simply represents the value of μ with $N_g = x$, and likewise for λ .

Systems of equations such as these, which describe the evolution of a probability distribution with time, are known as *Kolmogorov equations*. We will have a system of $A+1$ equations, one for each possible value of N_g .

There are a couple of special cases to address, namely, the cases of $\dot{p}_0$ and $\dot{p}_A$. Since $\lambda(0) = \mu(0) = 0$ and $p_{-1} = p_{A+1} = 0$:

$$\dot{p}_0(t) = p_1(t)\mu(1)$$

$$\dot{p}_A(t) = -p_A(t)(\lambda(A) + \mu(A)) + p_{A-1}(t)\mu(A-1)$$

We'll wrap up the system into vector form for convenient expression:

$$\mathbf{p}(t) = [p_0(t), p_1(t), p_2(t) \dots p_A(t)]$$

$$\dot{\mathbf{p}}(t) = \frac{d\mathbf{p}}{dt} = [\dot{p}_0(t), \dot{p}_1(t), \dot{p}_2(t) \dots \dot{p}_A(t)]$$

$$\dot{\mathbf{p}}(t) = \mathbf{p}(t)\mathbf{R}$$

$$\mathbf{R} = \begin{bmatrix} r_{0,0} & r_{0,1} & r_{0,2} & \cdots & r_{0,A} \\ r_{1,0} & r_{1,1} & r_{1,2} & \cdots & r_{1,A} \\ r_{2,0} & r_{2,1} & r_{2,2} & \cdots & r_{2,A} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ r_{A,0} & r_{A,1} & r_{A,2} & \cdots & r_{A,A} \end{bmatrix}$$

Figure 4 Evolution of p_x distribution for a pasture of size $A = 100$, for (a) $N = 40$ and (b) $N = 70$.

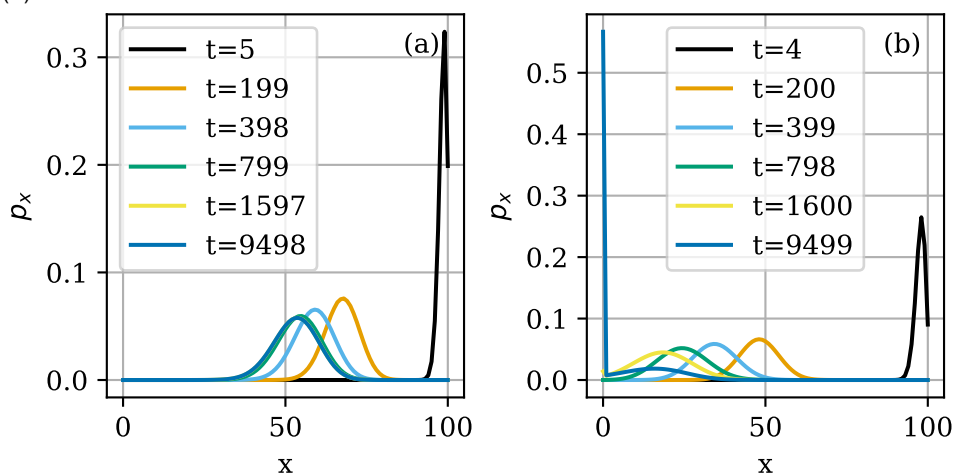
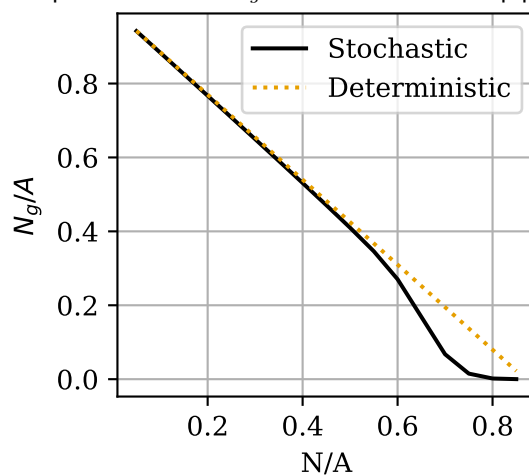


Figure 5 expected value of N_g at $t = 10000$ vs. sheep population.



$$r_{i,j} = \begin{cases} r_{0,j} = 0 \\ r_{i,i+1} = \lambda(i) \\ r_{i+1,i} = \mu(i) \\ r_{i,i} = -(\lambda(i) + \mu(i)) \end{cases}$$

All that's left to do now is to pick a value for N and an initial value for N_g and solve. This is most easily done numerically, as has been done here using the Runge-Kutta method.

Discussion

Limitations and potential improvements This is a continuous-time model. The process of developing this model involves considering a change in p_x from time t to

time $t + \Delta t$ and taking the limit as $\Delta t \rightarrow 0$. In the process of taking this limit, the probability of multiple events occurring in time Δt (e.g. multiple sheep eating or multiple grass blocks regrowing) approaches zero. A continuous time model is appropriate for real-world ecology, but the physics engine of *Minecraft* operates in discretized time intervals, called *ticks*, of 0.05 seconds. Taking the limit of $\Delta t \rightarrow 0$ therefore loses out on the very real possibility of multiple events occurring in a single tick.

Unsimplified model The physics engine of *Minecraft* operates in time intervals of 0.05 seconds. Therefore the model discussed here will use discrete time intervals $\Delta t = 0.05$

Within time Δt , a sheep has a chance $\frac{RN_g}{A} \Delta t$ to eat. For a pasture with N sheep, the number of sheep that eat in time Δt is a binomial random variable of N trials and $\frac{RN_g}{A} \Delta t$ success probability.

The law of rare events states that a binomial distribution with a large number of events n and a small probability of success p can be approximated by a Poisson distribution with expected value np . We can use this to replace the above binomial distribution with a simpler Poisson distribution, with expected value $\frac{NRN_g}{A} \Delta t$.

Similarly, we can model grass growth with a Poisson distribution. The probability that the grass population changes by an amount $\Delta_r N_g$ in time Δt due to grass regrowth is a poisson distribution with expected value $\dot{N}_g \Delta t = kN_g \left(1 - \frac{N_g}{A}\right) \Delta t$, as given by (2).

If n_r is the number of grass blocks that regrew, and n_e is the number that were eaten by sheep, the overall change in N_g is given by:

$$\Delta N_g = n_r - n_e$$

The probability distribution of the difference of two Poisson-distributed variables n_1 and n_2 , with expected values m_1 and m_2 , is given by the Skellam distribution [2]:

$$\Pr(n_1 - n_2 = x) = e^{-(m_1+m_2)} \left(\frac{m_1}{m_2}\right)^{\frac{x}{2}} I_x(2\sqrt{m_1 m_2})$$

Where I_x is a modified bessel function of the first kind of order x .

Therefore, ΔN_g is dictated by a Skellam distribution with

$$\mu_1 = kN_g \left(1 - \frac{N_g}{A}\right) \Delta t$$

$$\mu_2 = \frac{NRN_g}{A} \Delta t$$

Let's consider a pasture at $N_g = x$ at time t_0 , i.e., $N_g(t_0) = x$. The probability that the pasture ends up in the state $N_g(t + \Delta t) = x + \Delta N_g$ is given by the probability of a change ΔN_g times the probability of the state $N_g(t_0) = x$.

However, the pasture may arrive at the same value of N_g through multiple paths.

For example, All of the following paths would lead to $N_g(t + \Delta t) = 100$:

- $N_g(t_0) = 100$ and $\Delta N_g = 0$
- $N_g(t_0) = 101$ and $\Delta N_g = -1$
- $N_g(t_0) = 102$ and $\Delta N_g = -2$
- $N_g(t_0) = 99$ and $\Delta N_g = +1$
- ...
- $N_g(t_0) = 100 + n$ and $\Delta N_g = -n$

And so on. The probability of each path is the probability of the initial state existing, times the probability of the value of ΔN_g . In general the probability of the state $N_g(t_0 + \Delta t) = x + n$ is the sum of the probabilities of all the paths that lead there, i.e. paths for which the initial state is $N_g(t_0) = x$ and $\Delta N_g = n$, for any value of n :

$$\Pr(N_g(t_0 + \Delta t) = x + n) = \sum_n \Pr(N_g(t_0) = x) \Pr(\Delta N_g = n)$$

This looks like a convolution of the distribution of N_g with the distribution of ΔN_g . However, this is not a simple convolution, as the distribution of ΔN_g is dependent on the value of N_g .

Although this effort is becoming mathematically intractable, a numerical solution is still well within the realm of possibility. If the distribution of N_g is known at $t = t_0$, the probability of all possible paths can be calculated to get the distribution of N_g at $t = t_0 + \Delta t$, and then for $t = t_0 + 2\Delta t$, $t = t_0 + 3\Delta t$, and so on.

REFERENCES

1. Matis, J., Kiffe, T. Stochastic Population Models: a Compartmental Perspective.
2. Skellam, J. G. (1946). The Frequency Distribution of the Difference Between Two Poisson Variates Belonging to Different Populations.